
SOFTWARE DESIGN

for

Encost Smart Graph Project

Version 1.0

Prepared by: **Min Soe Htut**
SoftFlux Engineer

SoftFlux

March 28, 2025

Contents

1	Introduction/Purpose	3
1.1	Purpose	3
1.2	Document Conventions	3
1.3	Intended Audience and Reading Suggestions	4
1.4	Project Scope	4
2	Specialized Requirements Specification	5
3	Software Architecture	5
3.1	Component Diagram	6
3.2	Deployment Diagram	7
3.3	Process Flow Diagram	8
3.4	Software and Data Maintenance Considerations	9
4	Component Design	9
4.1	Class Diagram	10
4.1.1	User	10
4.1.2	DatasetHandler	11
4.1.3	Device	11
4.1.4	StatisticsReport	12
4.1.5	GraphRenderer	12
4.1.6	GraphStream Library	13
4.2	User Interface Design	13
4.2.1	Welcome Screen	14
4.2.2	Community User - Graph Access	15
4.2.3	Encost User - Username Prompt	16
4.2.4	Encost User - Password Prompt	17
4.2.5	Encost Feature Menu	18
4.2.6	Custom Dataset Upload Prompt	19
4.2.7	Dataset Successfully Loaded	20
4.2.8	Graph View Output	21
4.2.9	Summary Statistics	22
4.3	Non-functional Requirements	22
4.3.1	Performance	23
4.3.2	Security	23
4.3.3	Reliability	23
4.3.4	Compatibility	23
4.3.5	Usability	23
4.3.6	Maintainability	23
5	Conclusion	24

Revision History

Name	Date	Reason for Changes	Version
Min Soe Htut	11 March 2024	Initial Draft	0.1
Min Soe Htut	15 March 2024	Draft 2 Specialized Requirements Specification	0.2
Min Soe Htut	18 March 2024	Draft 3 Software Architecture	0.3
Min Soe Htut	25 March 2024	Draft 4 Component Design	0.4
Min Soe Htut	28 March 2024	Finisize	1.0

1 Introduction/Purpose

1.1 Purpose

This document provides a comprehensive Software Requirements Specification (SRS) for the Encost Smart Graph Project (ESGP). It defines the intended functionality, system architecture overview, and interaction flow between components. This SRS serves as a foundational reference for developers, testers, and stakeholders, ensuring that all teams work toward a consistent and validated understanding of the project requirements.

1.2 Document Conventions

Throughout this document, the following conventions and terminology are used:

- **ESGP** – Encost Smart Graph Project
- **SRS** – Software Requirements Specification
- **UI** – User Interface
- **CLI** – Command-Line Interface (used for interaction in the ESGP system)
- **Device** – A smart home product recorded in the Encost dataset
- **User** – A person interacting with the ESGP system (either a community user or Encost user)

All file paths are assumed to follow standard local file system formats. Graph visualisations are generated using the GraphStream library.

1.3 Intended Audience and Reading Suggestions

This document is intended for software engineers, testers, users, and project managers involved in the development of ESGP. Each audience can benefit from specific sections of this SRS:

- **Software Engineers:** Can use the SRS to understand system functionality, user roles, and component interactions when designing or implementing system features.
- **Testers:** Will refer to the documented requirements to develop appropriate test cases and validate system functionality.
- **Users:** Can review this SRS to understand how the application will behave and what features are available to them based on their role.
- **Project Managers:** Can reference the SRS to monitor requirement completeness, identify feature scope, and ensure the system aligns with business goals.

1.4 Project Scope

The Encost Smart Graph Project (ESGP) is a software system designed to visualize the usage and connectivity of Encost’s smart home devices within households across New Zealand. By utilizing a graph-based representation, the system allows users to explore device relationships, understand network structures, and analyze connectivity trends. Community users can access predefined visualizations based on the Encost Smart Homes Dataset, while Encost users have additional privileges, including loading custom datasets and retrieving summary statistics on device distribution, location, and connectivity. The ESGP system is designed to be user-friendly, reliable, and secure, ensuring seamless data analysis for both technical and non-technical users. Ultimately, ESGP

aims to provide Encost with deeper insights into device adoption and network behavior to support informed business decisions.

2 Specialized Requirements Specification

While the Encost Smart Graph Project (ESGP) is primarily described as a local application, some deployment and data handling details were not explicitly defined in the SRS. It has been clarified by the client that Encost will contact community users directly with installation instructions; the specific method of distribution (e.g., via website, email, or installer) is left to developer discretion. Additionally, although the SRS mentions daily backups of the source code, it does not specify where or how these backups should be stored. This can be addressed by implementing automated backups either to a secure cloud service or a designated local server. Furthermore, while the SRS requires that all datasets be anonymised, it does not define how anonymisation should be performed. It has been confirmed that developers can assume all datasets are already anonymised before being processed by ESGP, and therefore, no additional verification is required within the application. Clarifying these aspects helps ensure the SDS aligns closely with both project expectations and practical implementation choices.

3 Software Architecture

The software architecture of the Encost Smart Graph Project (ESGP) defines the high-level structure of the system, focusing on component interactions, user flow, and system responsibilities. The ESGP system is designed as a modular console-based Java application that operates entirely offline. Its architecture promotes the separation of concerns through distinct components responsible for user authentication, dataset management, graph rendering, and statistical analysis.

This chapter presents a set of diagrams to visualise the architectural decisions made during the design process. These include component, deployment, process flow, and sequence diagrams. Each diagram highlights different architectural aspects, such as how users interact with the system, how internal modules communicate, and how data flows through the application.

The ESGP architecture is purposefully simple and self-contained to align with its offline operational requirement. This improves reliability, simplifies testing, and reduces external dependencies. The design also ensures the system is maintainable, scalable for future features, and accessible across various platforms supporting the Java runtime

environment.

3.1 Component Diagram

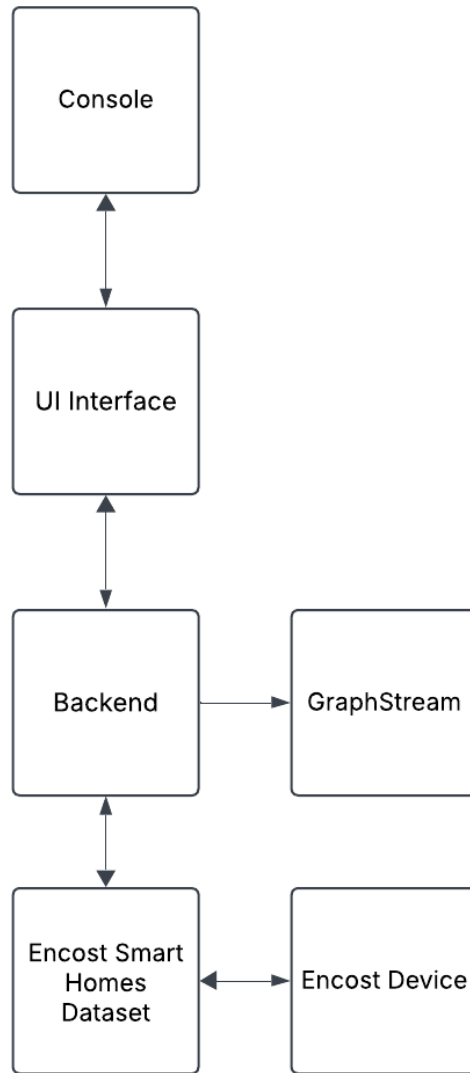


Figure 3.1: Component Diagram of the ESGP System

The ESGP system is composed of several key components that work together to deliver the graph-based visualization functionality. Figure 3.1 illustrates the main components and their interactions. The system begins with user interaction through a Console,

which handles all command-line prompts and inputs. This communicates directly with the UI Interface, which serves as the intermediary between user actions and the system logic. The Backend processes inputs, handles user authentication, loads datasets, and manages requests to generate visualizations. It interacts with the GraphStream library to create and render graph structures from the dataset. The Backend also interfaces with the Encost Smart Homes Dataset, which contains device data collected from New Zealand homes. This dataset is constructed from interactions with various Encost Devices. These devices provide data on connectivity and usage, which is continuously updated in the dataset. The modularity of this design ensures a clear separation of concerns and supports scalability, allowing each component to evolve independently.

3.2 Deployment Diagram

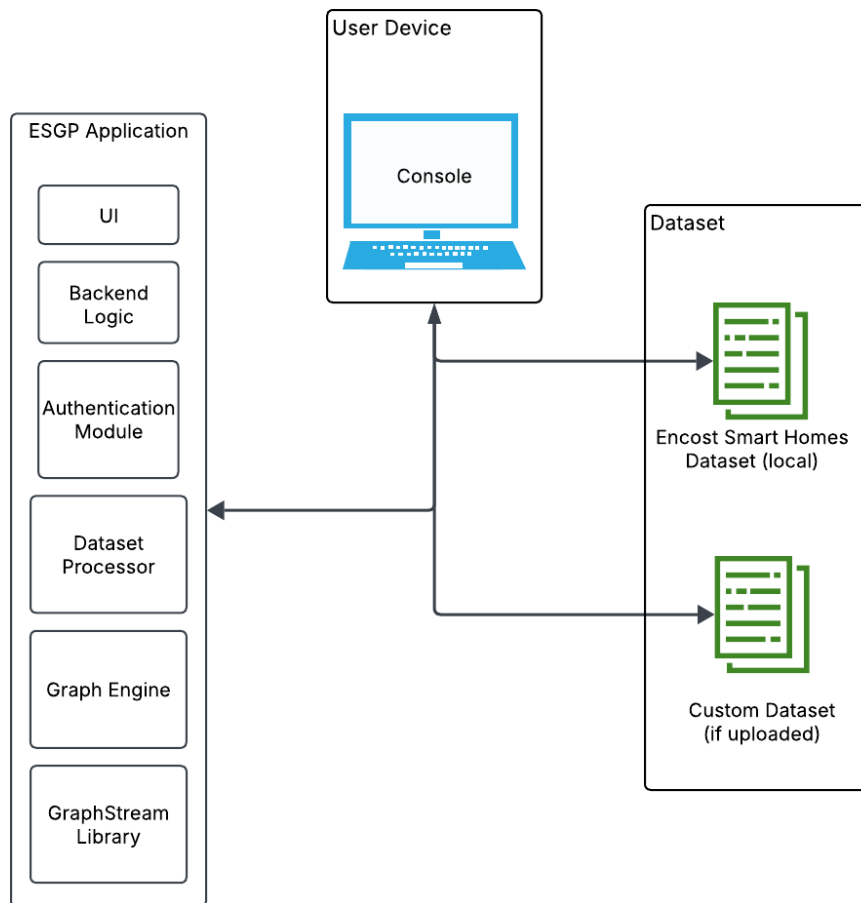


Figure 3.2: Local Deployment of ESGP System

The deployment diagram illustrates the architecture of the ESGP system as a standalone desktop application. ESGP is designed to run entirely on a user’s local machine, with no need for internet connectivity or communication with external servers. All core components—including the user interface, backend logic, authentication module, dataset processor, and graph engine—are packaged within the ESGP application and operate locally. Users interact with the system via a console-based interface, which facilitates data input and visualisation display. The application accesses the Encost Smart Homes Dataset stored locally, and Encost users may optionally upload their own custom datasets for analysis.

3.3 Process Flow Diagram

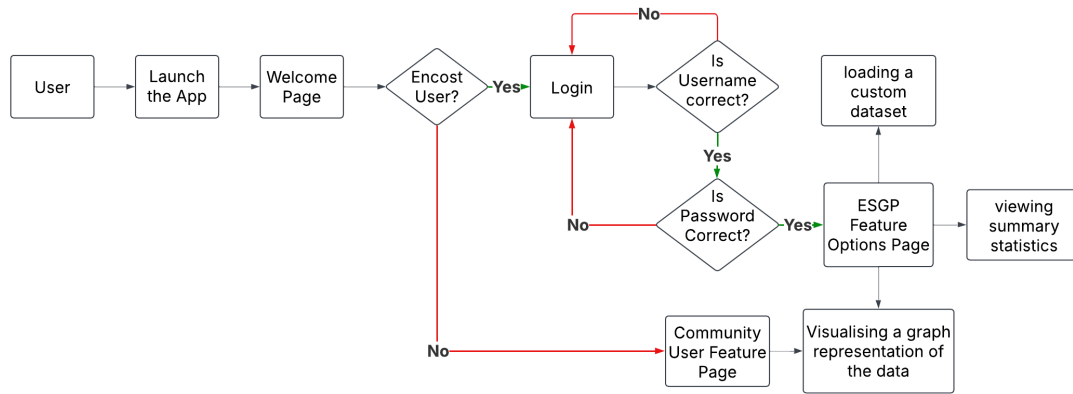


Figure 3.3: Process Flow of User Interaction with ESGP

Process flow for users interacting with the ESGP system. Upon launching the application, users are directed to a welcome page where they identify their user type. If the user is an Encost user, they are prompted to log in. The system then verifies the entered credentials through a two-step authentication process: username and password validation. Once successfully authenticated, Encost users are granted access to the full set of ESGP features, including loading custom datasets, viewing summary statistics, and graph visualisation.

Community users bypass authentication and are directed straight to a simplified interface where they can only view the preloaded graph visualisation from the Encost Smart Homes Dataset. This conditional flow ensures that users are granted access to features appropriate to their role while maintaining usability and security within a local deployment context.

3.4 Software and Data Maintenance Considerations

In a local offline environment, software maintenance and update distribution present unique challenges. ESGP does not support online updates, so users must manually install newer versions. To support this, the application will clearly display its version and include update instructions in its documentation. Changelogs and release notes will accompany future versions to assist users in identifying new features or critical fixes.

Additionally, long-term usage of ESGP may result in the accumulation of user-generated data such as custom datasets. To maintain performance and avoid storage issues, the system encourages proper data hygiene. Uploaded datasets are stored in a separate directory, and users may delete or archive old data when needed. Input validation is applied on each dataset upload to ensure compatibility with the ESGP engine, minimizing the risk of errors caused by corrupted or outdated files.

To support better portability and collaboration, ESGP will also provide features for exporting and importing datasets. This allows users to back up their data externally, transfer datasets between systems, or share data files with other users if needed. Supported formats such as CSV or JSON ensure compatibility with other tools and simplify long-term dataset management. Since ESGP does not use any cloud services or online backups, these export/import features serve as an essential manual safeguard for preserving important data.

By addressing both software and data maintenance explicitly, ESGP remains sustainable and reliable even without centralized infrastructure or network-based update systems.

4 Component Design

The Encost Smart Graph Project (ESGP) is composed of several modular components, each responsible for a distinct aspect of system functionality. The user management component handles login and access control, distinguishing between community and Encost users. Dataset handling is managed by a dedicated component that validates input files and loads device information. Visualisation is performed using a graph renderer that integrates with the GraphStream library to display connectivity between devices. Statistical analysis is handled by a reporting component that computes device distribution and usage metrics. All user interactions occur through a console-based interface designed for clarity and ease of use. This separation of concerns enhances maintainability, testing, and future extensibility of the system.

4.1 Class Diagram

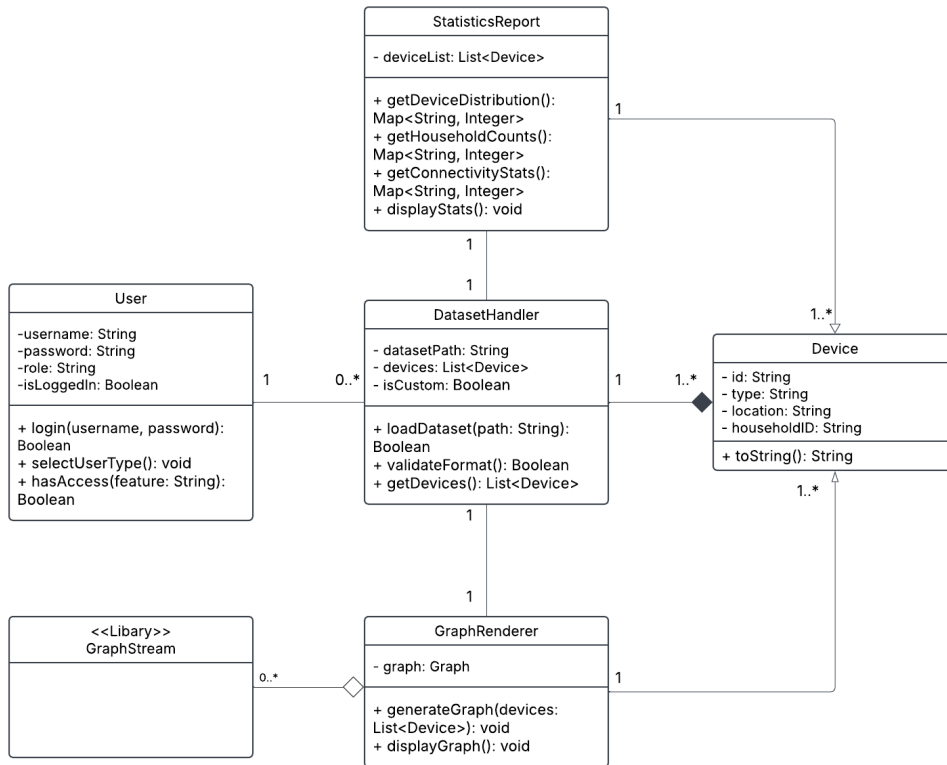


Figure 4.1: UML Class Diagram for ESGP System

Figure 4.1 presents the UML class diagram of the ESGP system. The system is composed of modular components that follow a clear separation of concerns. Users interact with the system through the **User** class, which manages authentication and access control. The **DatasetHandler** loads and validates datasets, storing parsed **Device** objects. These devices are used by both the **StatisticsReport** class to generate summary statistics and the **GraphRenderer** class to build a visual graph representation. The **GraphStream** library supports graph layout and rendering, while the system ensures structured data flow and maintainability through clearly defined associations.

4.1.1 User

The **User** class manages authentication and feature access for different types of users in the ESGP system. It stores login credentials and determines which system features a user is allowed to use based on their role.

Attributes:

- `username: String` – Stores the username entered by the user.
- `password: String` – Stores the password for authentication.
- `role: String` – Defines the user type, such as "community" or "encost".
- `isLoggedIn: Boolean` – Indicates whether the user has successfully logged in.

Methods:

- `login(username, password): Boolean` – Checks the given credentials and returns true if valid.
- `selectUserType(): void` – Assigns the correct user role based on input.
- `hasAccess(feature): Boolean` – Returns true if the current user has permission to use the specified feature.

4.1.2 DatasetHandler

The `DatasetHandler` class is responsible for managing and validating datasets within the ESGP system. It loads datasets from a specified path, verifies their format, and stores the devices contained within for further processing.

Attributes:

- `datasetPath: String` – Stores the file path from which the dataset is loaded.
- `devices: List<Device>` – Contains the list of devices parsed from the dataset.
- `isCustom: Boolean` – Indicates whether the dataset was user-uploaded (custom) or the default Encost dataset.

Methods:

- `loadDataset(path: String): Boolean` – Loads a dataset from the specified file path and returns true if successful.
- `validateFormat(): Boolean` – Verifies whether the dataset file conforms to the expected format.
- `getDevices(): List<Device>` – Returns the list of devices loaded into the dataset.

4.1.3 Device

The `Device` class represents a single smart device in the Encost dataset. Each device contains metadata such as its type, physical location, and the household it belongs to. This class is used across multiple components of the ESGP system for statistics calculation, graph rendering, and dataset parsing.

Attributes:

- **id:** `String` – A unique identifier for the device.
- **type:** `String` – The type of the device (e.g., light, thermostat, camera).
- **location:** `String` – The physical location of the device within the household (e.g., kitchen, bedroom).
- **householdID:** `String` – The identifier for the household that owns the device.

Method:

- **toString():** `String` – Returns a string representation of the device object, typically for display or logging purposes.

4.1.4 StatisticsReport

The `StatisticsReport` class is responsible for computing and storing summary statistics based on the dataset of devices. It provides the core functionality for generating insights such as device distribution, household usage, and connectivity trends. These statistics are used to inform users about patterns within the dataset and support visualisation in the ESGP system.

Attribute:

- **deviceList:** `List<Device>` – A list of `Device` objects used as the basis for all statistical calculations.

Methods:

- **getDeviceDistribution():** `Map<String, Integer>` – Returns the count of devices per device type.
- **getHouseholdCounts():** `Map<String, Integer>` – Returns the number of devices per household.
- **getConnectivityStats():** `Map<String, Integer>` – Returns statistics related to device interconnectivity (e.g., average connections per household).
- **displayStats():** `void` – Prints or displays the calculated statistics in a readable format for the user.

4.1.5 GraphRenderer

The `GraphRenderer` class is responsible for transforming the device dataset into a visual representation using graph-based techniques. It interacts with the `GraphStream` library to create, layout, and display the graph. This component ensures that device relationships (e.g., connectivity between smart devices) are presented in an intuitive and meaningful way for the user.

Attributes:

- **graph:** `Graph` – A reference to the `Graph` object generated from the dataset, managed by the `GraphStream` library.

Methods:

- **generateGraph(devices: List<Device>): void** – Generates the graph structure based on the list of `Device` objects, defining nodes and edges according to their connectivity or shared properties.
- **displayGraph(): void** – Displays the graph in a GUI window using `GraphStream`'s rendering capabilities.

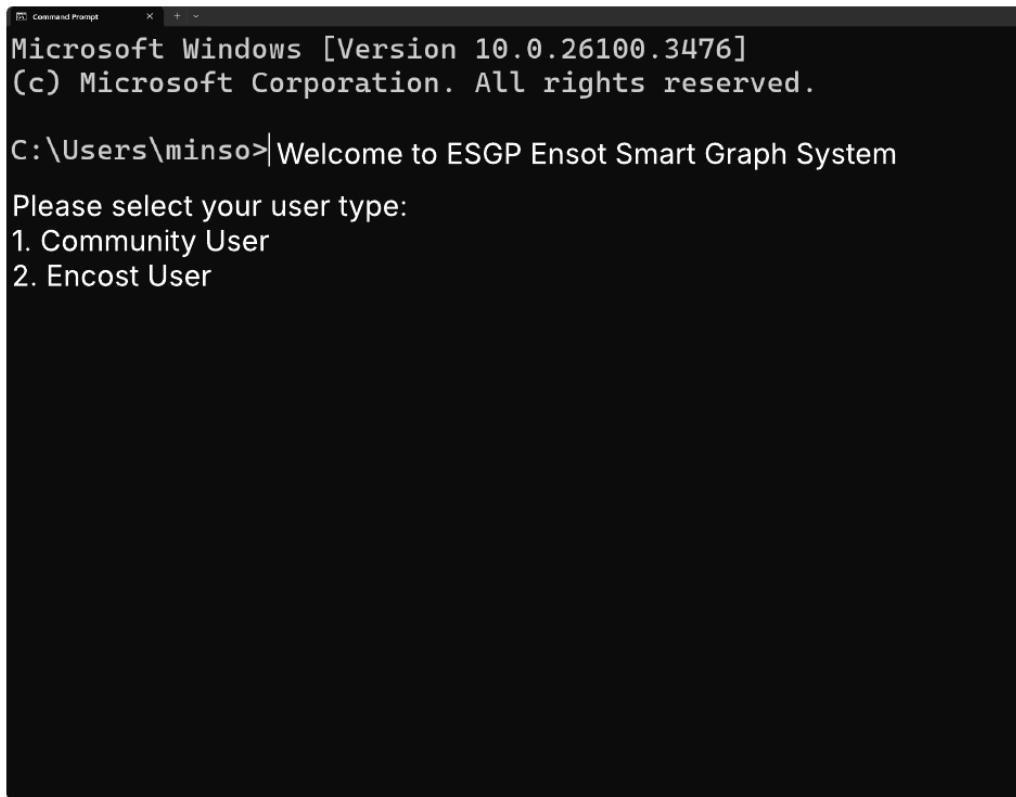
4.1.6 GraphStream Library

ESGP uses the external `GraphStream` library for graph visualisation. It provides rendering and layout tools that the `GraphRenderer` class integrates to display interactive visual graphs of device networks.

4.2 User Interface Design

The ESGP system is designed as a console-based application to maximize compatibility and reduce system overhead. Each interaction takes place through clearly structured text prompts. This approach ensures the system works on any machine without requiring a graphical user interface. Below are screenshots and descriptions of the user journey from start to finish.

4.2.1 Welcome Screen

A screenshot of a Windows Command Prompt window. The title bar reads "Command Prompt". The text inside the window is as follows:
Microsoft Windows [Version 10.0.26100.3476]
(c) Microsoft Corporation. All rights reserved.

C:\Users\minso>Welcome to ESGP Ensot Smart Graph System

Please select your user type:
1. Community User
2. Encost User

Figure 4.2: User Type Selection Screen

When the user launches ESGP, they are greeted with a welcome message prompting them to choose between "Community user" or "Encost user". This decision determines the level of access they will receive. Community users skip authentication and access limited functionality, while Encost users are required to log in for full-feature access.

4.2.2 Community User - Graph Access

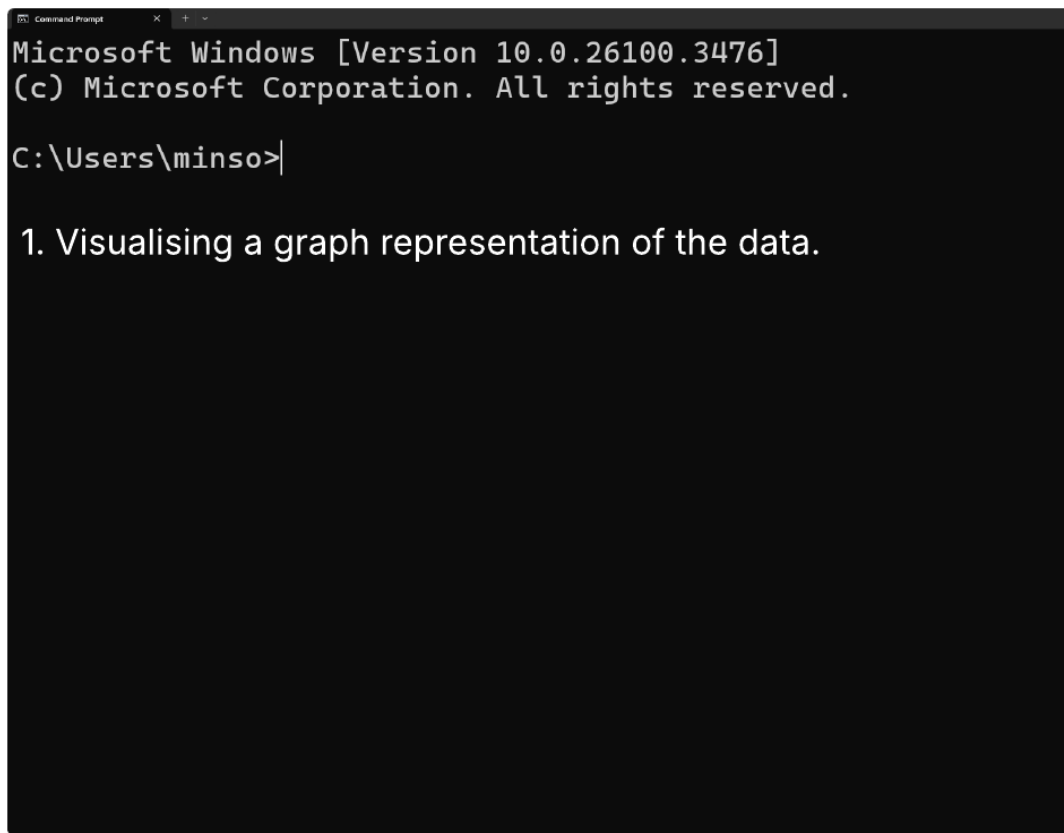


Figure 4.3: Community User Graph Option

Community users are presented with a simple, one-option menu that allows them to visualise a predefined graph of the Encost Smart Homes Dataset. This mode is read-only, preventing unauthorized modification or data access.

4.2.3 Encost User - Username Prompt

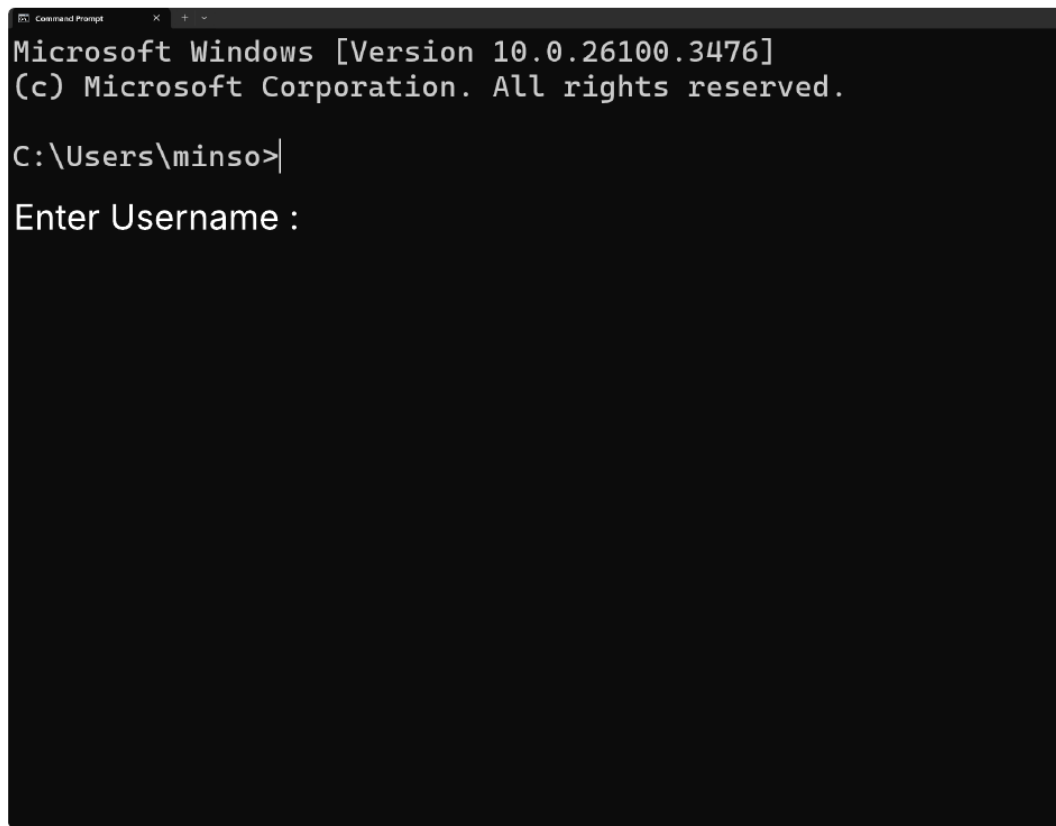


Figure 4.4: Username Input for Encost User

Encost users begin their login process by entering a username. If the username is valid and found in the system, the user proceeds to the password prompt. Otherwise, the system displays an error and re-prompts for input.

4.2.4 Encost User - Password Prompt

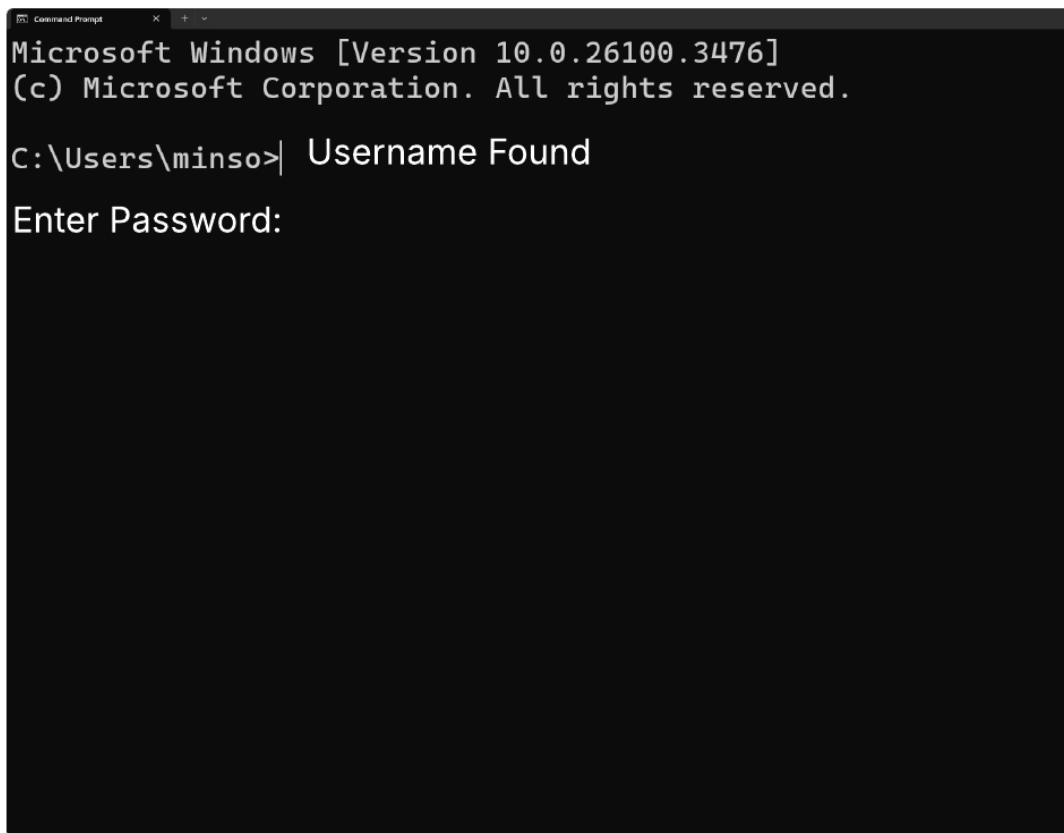


Figure 4.5: Password Input After Username is Found

Once a valid username is confirmed, the user is prompted to enter their password. If the entered password matches the expected value, the login process is successful and the user gains access to the full feature set.

4.2.5 Encost Feature Menu

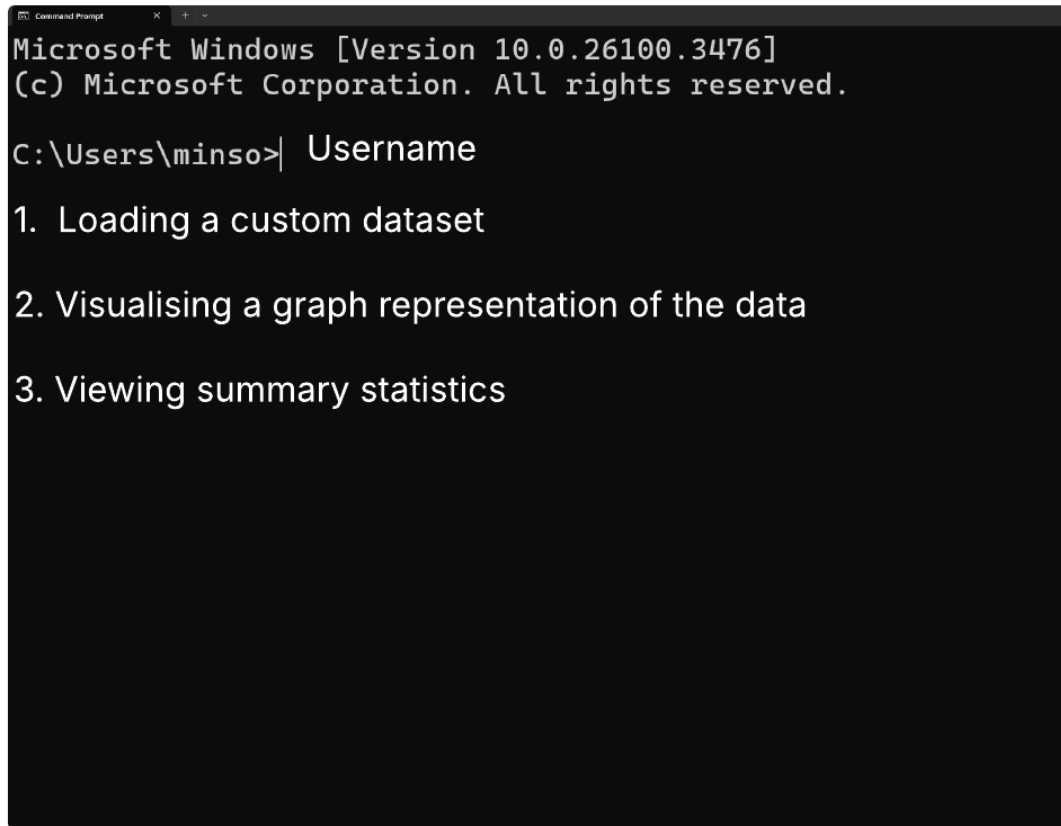


Figure 4.6: Feature Selection for Encost Users

After login, Encost users are presented with a menu containing all available features. They can choose to load a custom dataset, view a graph representation, or explore summary statistics based on the currently active dataset.

4.2.6 Custom Dataset Upload Prompt

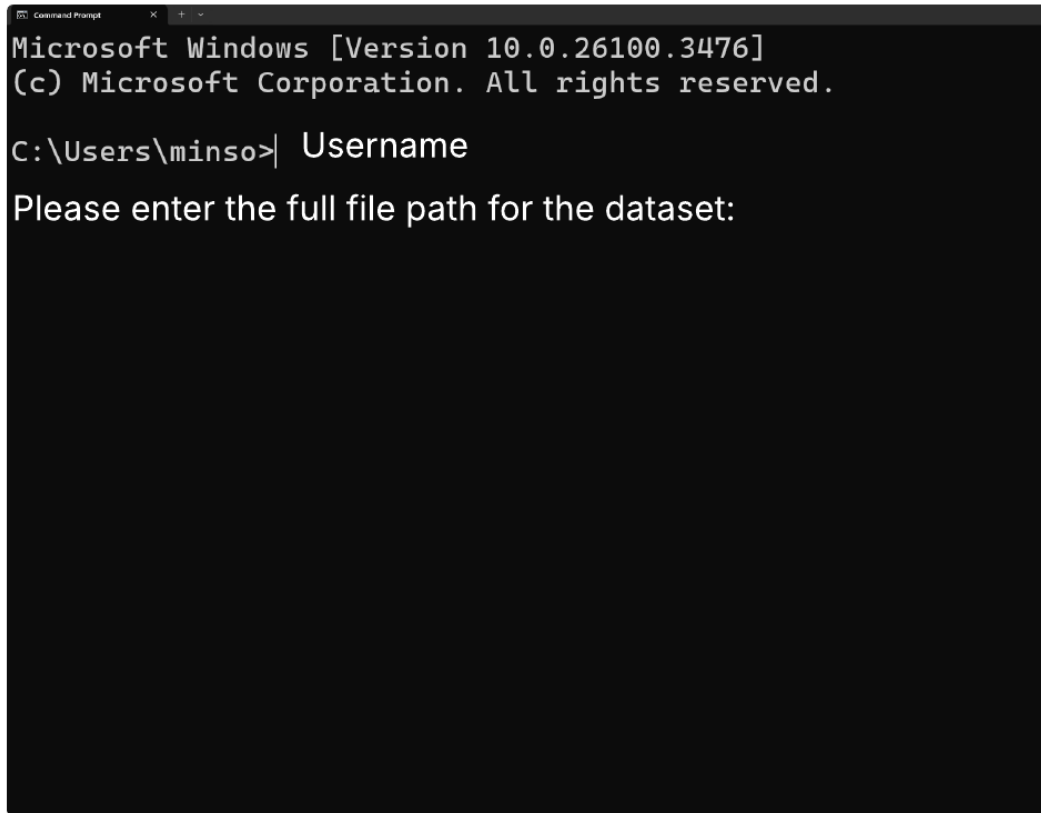
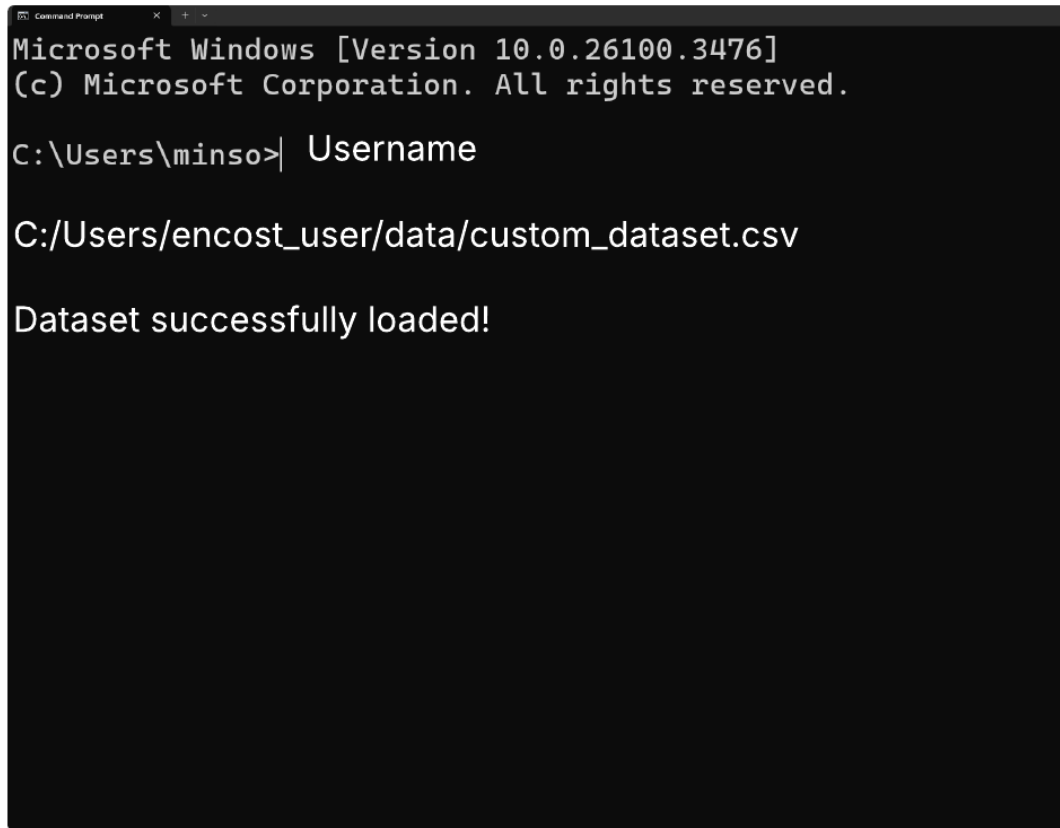


Figure 4.7: Prompt to Enter File Path for Custom Dataset

If the user selects the dataset upload option, they are prompted to enter the full file path of the dataset. The system checks whether the file exists and validates its format before proceeding with the load.

4.2.7 Dataset Successfully Loaded



```
Microsoft Windows [Version 10.0.26100.3476]
(c) Microsoft Corporation. All rights reserved.

C:\Users\minso> Username

C:/Users/encost_user/data/custom_dataset.csv

Dataset successfully loaded!
```

Figure 4.8: Confirmation Message for Successful Dataset Load

After validating and parsing the dataset, the system displays a success message confirming that the new dataset has been loaded. At this point, other options like graph visualisation and statistics will now use this custom dataset.

4.2.8 Graph View Output

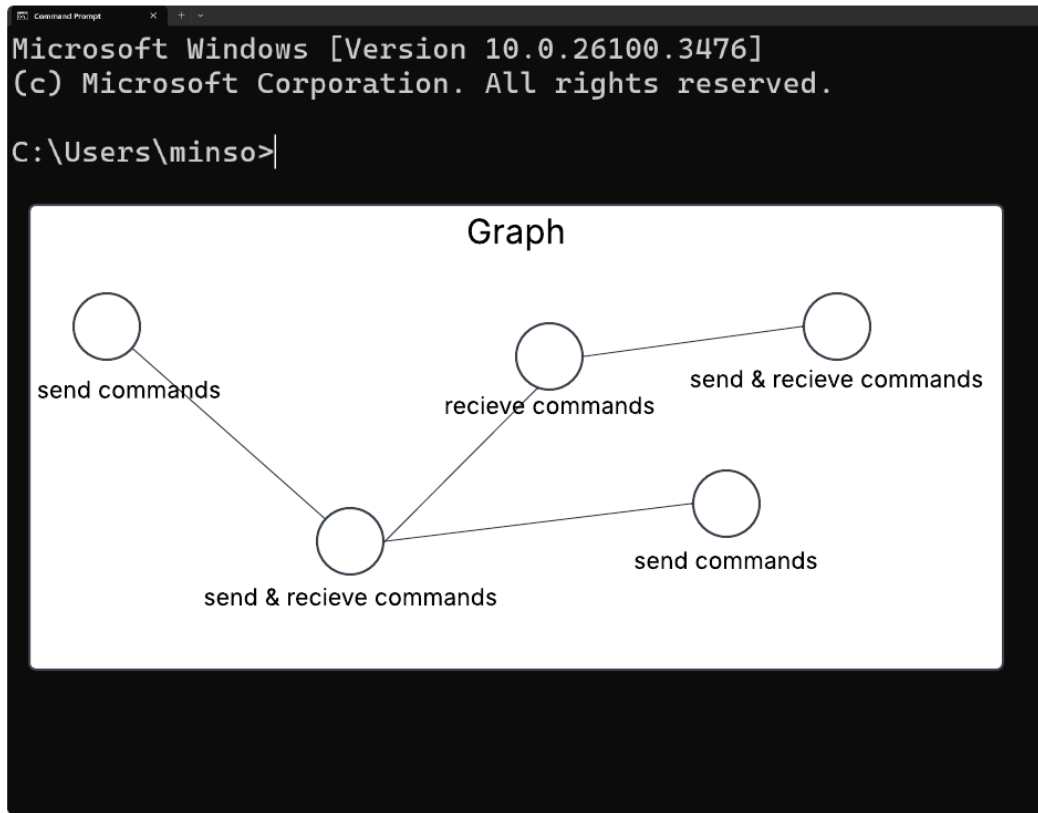
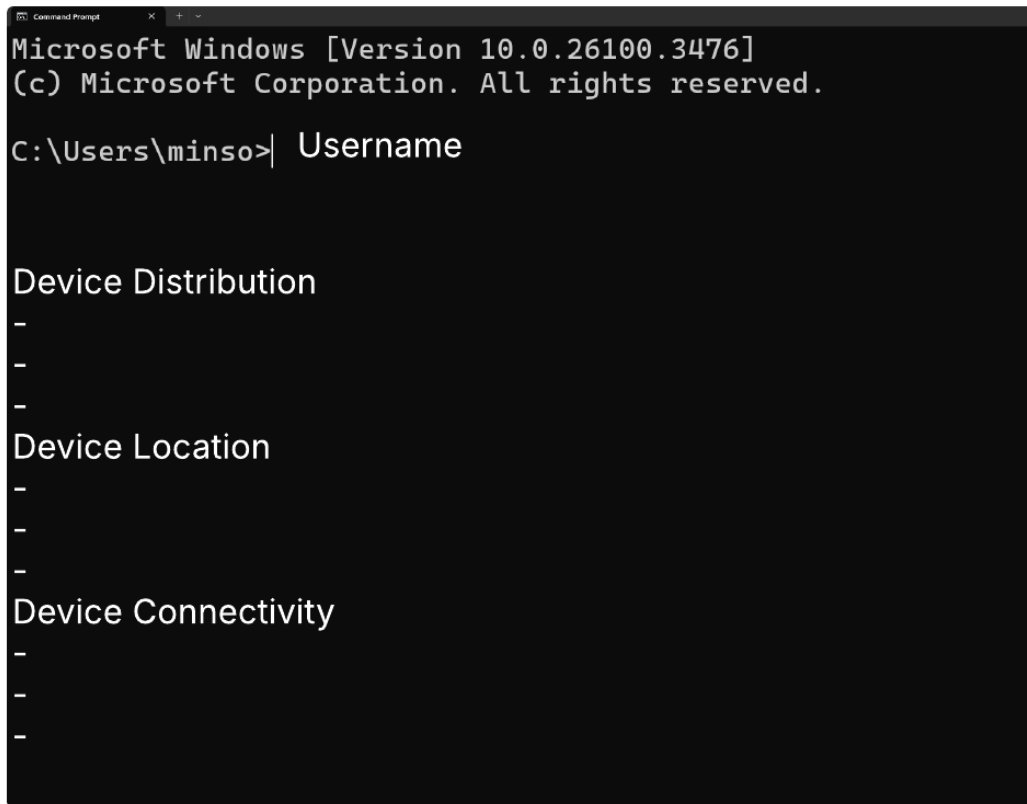


Figure 4.9: Graph Visualisation Rendered via GraphStream

When the user selects to visualise the dataset, the system launches a graphical window using the GraphStream library. The resulting graph shows devices as nodes and their connections as edges, providing a clear visual summary of the smart device network.

4.2.9 Summary Statistics



```
Microsoft Windows [Version 10.0.26100.3476]
(c) Microsoft Corporation. All rights reserved.

C:\Users\minso> Username

Device Distribution
-
-
-
Device Location
-
-
-
Device Connectivity
-
-
-
```

Figure 4.10: Options for Viewing Summary Statistics

The interface presents a statistics to Encost users. Users can view the summer of device distribution, device location, and device connectivity. The system then prints the relevant results in a well-formatted console output.

4.3 Non-functional Requirements

In addition to the functional requirements, the ESGP system must meet several non-functional expectations to ensure usability, security, performance, and long-term reliability. These qualities support a seamless user experience and ensure that the system operates efficiently in a variety of environments.

4.3.1 Performance

- The system should respond to user inputs promptly, with minimal delay across all operations including loading datasets, displaying statistics, and generating graphs.
- It must handle large datasets efficiently, ensuring smooth performance without lag or memory issues on standard hardware.

4.3.2 Security

- Although ESGP is a local application, it must ensure the protection of user data such as login credentials and datasets.
- User roles (e.g., community or Encost) must be verified through authentication before granting access to restricted features.
- Credentials should be stored securely, and access to custom datasets should be limited to authenticated users.

4.3.3 Reliability

- The system must operate reliably in an offline environment, with minimal risk of data loss or unexpected crashes.
- Dataset validation and error handling should be implemented to avoid corrupted inputs or unexpected behavior.

4.3.4 Compatibility

- ESGP should function on multiple platforms, including Windows, macOS, and Linux systems that support Java.
- No dependencies on internet access or web-based services should be assumed.

4.3.5 Usability

- The command-line interface should be clean, intuitive, and guide the user clearly through each step of the system.
- Clear feedback and error messages should be presented to help users understand the result of their actions or correct any mistakes.

4.3.6 Maintainability

- The system should support easy updates by packaging all application logic into a modular structure.
- Source code should be well-documented to facilitate updates or debugging by other developers in the future.

5 Conclusion

This Software Requirements Specification (SRS) document presents the structural and behavioural requirements of the Encost Smart Graph Project (ESGP). It outlines the system’s architecture, component responsibilities, user interface design, and both functional and non-functional considerations.

Key system components—including user management, dataset handling, visualisation, and statistics generation—are described through diagrams and structured documentation. The console-based user interface is tailored for both community and Encost users, ensuring accessibility and security based on user roles.

Multiple design representations, such as component, deployment, sequence, and process flow diagrams, support future development and system comprehension. Emphasis is placed on offline reliability, modularity, and clear user interaction design.

By adhering to this SRS, the development process remains aligned with client expectations and software engineering standards, ensuring that ESGP is robust, maintainable, and user-focused.